

Database Management System

Design and Implementation of a Database Management System for Hostel Accommodation and Student Services



Submitted by:

Name: Vallamkondu Sai Jaideep

Roll no: 25WU0101149

Specialization: AIML

Section: Whales

Submitted to: Dr. Kiranmayee Adavala

(B. Tech 2025-29)

School of Technology

Submitted on: 07/10/2026

Contents

1. CoverPage
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2. Abstract

Managing a hostel on paper registers or scattered spreadsheets leads to duplicate entries, room over-allocation, lost fee records and slow complaint handling. This project presents **Hostel Accommodation and Student Services**, a web-based database admin panel that centralises the records of hostel staff, hostel blocks, rooms, students, fee payments, complaints, mess subscriptions and visitors.

The system is built on a normalised MySQL database of eight related tables connected through primary and foreign keys. The application is developed with Node.js (v24.15.0), run with `npm start`, and provides a browser interface with a sidebar for each module where an administrator can insert, view and delete records. Drop-down lists are filled from related tables so that only valid foreign-key values can be chosen. SQL joins, aggregates and subqueries generate reports such as room occupancy, block-wise strength, unpaid fees and open complaints.

The system reduces manual effort, improves data accuracy and gives wardens an up-to-date view of occupancy and finances.

3. Introduction and Problem Statement

A hostel houses many students across several blocks, and administrators must continuously track who stays in which room, which rooms are free, who has paid fees, which complaints are pending, which mess plan each student has taken and who visited whom. Doing this manually is error-prone and time-consuming.

Problem statement: To design and implement a database-driven web application that stores hostel data in a normalised relational database and lets authorised staff insert, view and delete records reliably, with fast retrieval of information through SQL queries.

Key problems addressed:

- Redundant and inconsistent student, room and block records.
- No structured link between wardens, blocks, rooms and students.
- Difficulty in finding students with pending or overdue fees.
- No central record of complaints, mess plans and visitor entries.

4. Objectives and Scope

Objectives

- Design a normalised relational schema (up to 3NF) for hostel management.
- Enforce integrity with primary keys, foreign keys and not-null constraints.
- Provide a web interface to insert, view and delete records in all eight modules.
- Provide analytical queries: occupancy, block-wise strength, fee defaulters, complaint status and mess revenue.
- Connect the Node.js application to MySQL reliably.

Scope

In scope: Staff, Hostel Block, Room, Student, Fee Payment, Complaint, Mess and Visitor modules, each with insert, view and delete operations, plus SQL reports.

Out of scope: user login and roles, online payment gateway, record editing from the UI, mobile application and automatic notifications.

5. Software and Hardware Requirements

Software

Component	Details
Operating system	Windows 10/11, macOS or Linux
Runtime	Node.js v24.15.0
Package manager	npm (application started with npm start)
Server	Node.js web server on port 3000
Database	MySQL (mysql command-line client and MySQL Server)
DB driver	mysql2
Front end	HTML, CSS and JavaScript admin panel with sidebar navigation
Tools	VS Code, MySQL command-line client / Workbench, Git, GitHub
Browser	Google Chrome / Edge / Firefox (latest)

Hardware

Component	Minimum requirement
Processor	Dual-core, 2 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	500 MB free space
Display	1366 x 768 or higher
Network	Not required at run time (localhost); internet only to install packages

6. ER Diagram

The ER diagram has eight entities. A Staff member (warden) manages hostel blocks, a block has many rooms, and a room houses many students. A student can have many fee payments, complaints, mess records and visitor entries.

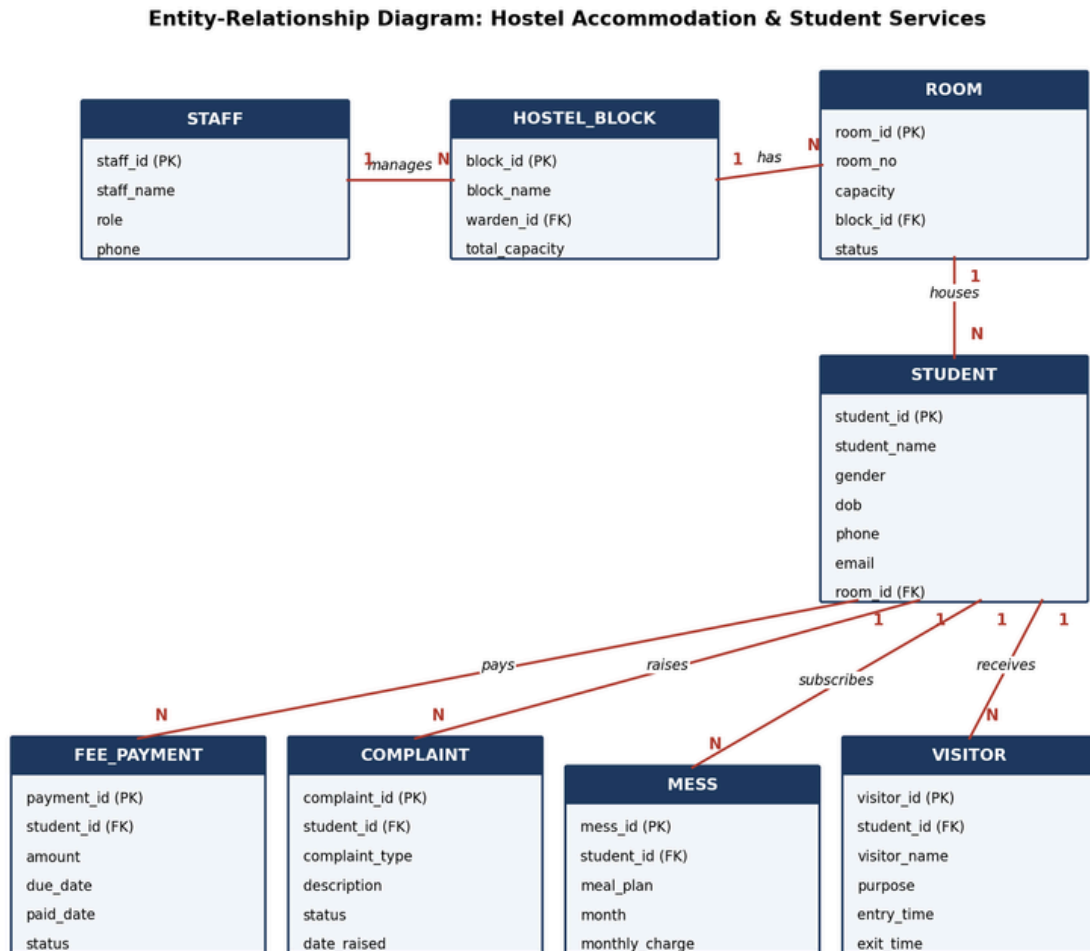


Figure 1: Entity-Relationship diagram

Cardinalities: Staff 1:N Hostel_Block, Hostel_Block 1:N Room, Room 1:N Student, Student 1:N Fee_Payment, Student 1:N Complaint, Student 1:N Mess, Student 1:N Visitor.

7. Relational Schema and Normalization

Relational schema

```
Staff(staff_id PK, staff_name, role, phone)
Hostel_Block(block_id PK, block_name, warden_id FK -> Staff, total_capacity)
Room(room_id PK, room_no, capacity, block_id FK -> Hostel_Block, status)
Student(student_id PK, student_name, gender, dob, phone, email,
        room_id FK -> Room)
Fee_Payment(payment_id PK, student_id FK -> Student, amount, due_date,
            paid_date, status)
Complaint(complaint_id PK, student_id FK -> Student, complaint_type,
          description, status, date_raised)
Mess(mess_id PK, student_id FK -> Student, meal_plan, month,
     monthly_charge)
Visitor(visitor_id PK, student_id FK -> Student, visitor_name, purpose,
        entry_time, exit_time)
```

Normalization

Unnormalised form (UNF). One large sheet holding student details with room number, block name, warden name and phone, and repeated rows for every fee payment, complaint, mess month and visitor.

1NF. All attributes are atomic and repeating groups (many payments, complaints, mess months and visitors per student) are moved into separate tables, each with a primary key.

2NF. No partial dependencies. Room details (room_no, capacity, status) depend only on room_id and are stored in Room, not repeated in Student. Block details depend only on block_id and are stored in Hostel_Block.

3NF. Transitive dependencies are removed. A student's block and warden are not stored in Student: Student references Room, Room references Hostel_Block, and Hostel_Block references the warden in Staff (student to room to block to warden). Warden name and phone therefore appear only once, in Staff.

Every non-key attribute in each table depends on the key, the whole key and nothing but the key, so the schema is in 3NF.

8. Data Dictionary

Staff

Column	Data type	Constraints	Description
staff_id	INT	PK, AUTO_INCREMENT	Staff identifier
staff_name	VARCHAR(100)	NOT NULL	Name of staff member
role	VARCHAR(50)	NOT NULL	Warden, Assistant Warden, etc.
phone	VARCHAR(15)		Contact number

Hostel_Block

Column	Data type	Constraints	Description
block_id	INT	PK, AUTO_INCREMENT	Block identifier
block_name	VARCHAR(50)	NOT NULL	Block name
warden_id	INT	FK to Staff(staff_id)	Warden in charge
total_capacity	INT		Total beds in the block

Room

Column	Data type	Constraints	Description
room_id	INT	PK	Room identifier
room_no	VARCHAR(10)	NOT NULL	Room number, e.g. A-101
capacity	INT	NOT NULL	Maximum occupants
block_id	INT	FK to Hostel_Block(block_id)	Block of the room
status	VARCHAR(20)		Available / Occupied

Student

Column	Data type	Constraints	Description
student_id	INT	PK, AUTO_INCREMENT	Student identifier
student_name	VARCHAR(100)	NOT NULL	Full name
gender	CHAR(1)		M / F
dob	DATE		Date of birth
phone	VARCHAR(15)		Contact number
email	VARCHAR(100)		Email address
room_id	INT	FK to Room(room_id)	Allotted room

Fee_Payment

Column	Data type	Constraints	Description
payment_id	INT	PK, AUTO_INCREMENT	Payment identifier
student_id	INT	FK to Student(student_id), NOT NULL	Paying student

Column	Data type	Constraints	Description
amount	DECIMAL(10,2)	NOT NULL	Fee amount
due_date	DATE		Due date
paid_date	DATE	NULL if unpaid	Date of payment
status	VARCHAR(20)		Paid / Pending / Overdue

Complaint

Column	Data type	Constraints	Description
complaint_id	INT	PK, AUTO_INCREMENT	Complaint identifier
student_id	INT	FK to Student(student_id), NOT NULL	Student who raised it
complaint_type	VARCHAR(50)		Electrical, Plumbing, etc.
description	VARCHAR(255)		Details
status	VARCHAR(20)		Pending / In Progress / Resolved
date_raised	DATE		Date raised

Mess

Column	Data type	Constraints	Description
mess_id	INT	PK, AUTO_INCREMENT	Mess record identifier
student_id	INT	FK to Student(student_id), NOT NULL	Student
meal_plan	VARCHAR(20)		Veg / Non-Veg
month	VARCHAR(7)		Month in YYYY-MM format
monthly_charge	DECIMAL(10,2)		Charge for the month

Visitor

Column	Data type	Constraints	Description
visitor_id	INT	PK, AUTO_INCREMENT	Visitor identifier
student_id	INT	FK to Student(student_id), NOT NULL	Student visited
visitor_name	VARCHAR(100)	NOT NULL	Visitor name
purpose	VARCHAR(100)		Purpose of visit
entry_time	DATETIME		Entry date and time
exit_time	DATETIME		Exit date and time

9. SQL Commands Used

9.1 DDL

```
CREATEDATABASE hostel_db;
USE hostel_db;

CREATE TABLE Staff (
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
    staff_name VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL,
    phone VARCHAR(15)
);

CREATE TABLE Hostel_Block (
    block_id INT AUTO_INCREMENT PRIMARY KEY,
    block_name VARCHAR(50) NOT NULL,
    warden_id INT,
    total_capacity INT,
    FOREIGN KEY (warden_id) REFERENCES Staff(staff_id)
);

CREATE TABLE Room (
    room_id INT PRIMARY KEY,
    room_no VARCHAR(10) NOT NULL,
    capacity INT NOT NULL,
    block_id INT,
    status VARCHAR(20),
    FOREIGN KEY (block_id) REFERENCES Hostel_Block(block_id)
);

CREATE TABLE Student (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    gender CHAR(1),
    dob DATE,
    phone VARCHAR(15),
    email VARCHAR(100),
    room_id INT,
    FOREIGN KEY (room_id) REFERENCES Room(room_id)
);

CREATE TABLE Fee_Payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    due_date DATE,
    paid_date DATE NULL,
    status VARCHAR(20),
    FOREIGN KEY (student_id) REFERENCES Student(student_id)
);

CREATE TABLE Complaint (
    complaint_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    complaint_type VARCHAR(50),
    description VARCHAR(255),
    status VARCHAR(20),
    date_raised DATE,
    FOREIGN KEY (student_id) REFERENCES Student(student_id)
);

CREATE TABLE Mess (
    mess_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
```

```
meal_plan VARCHAR(20),
month VARCHAR(7),
monthly_charge DECIMAL(10,2),
FOREIGN KEY (student_id) REFERENCES Student(student_id)
);
```

```
CREATE TABLE Visitor (
visitor_id INT AUTO INCREMENT PRIMARY KEY,
student_id INT NOT NULL,
visitor_name VARCHAR(100) NOT NULL,
purpose VARCHAR(100),
entry_time DATETIME,
exit_time DATETIME,
FOREIGN KEY (student_id) REFERENCES Student(student_id)
);
```

9.2 DML

```
INSERT INTO Staff (staff_name, role, phone) VALUES
('Ramesh Kumar', 'Warden', '9876500001'),
('Sunita Rao', 'Warden', '9876500002'),
('Anil Verma', 'Assistant Warden', '9876500003');
```

```
INSERT INTO Hostel_Block (block_name, warden_id, total_capacity) VALUES
('Block A', 1, 60), ('Block B', 2, 40);
```

```
INSERT INTO Room (room_id, room_no, capacity, block_id, status) VALUES
(101, 'A-101', 2, 1, 'Occupied'), (103, 'A-103', 2, 1, 'Available');
```

```
INSERT INTO Student (student_name, gender, dob, phone, email, room_id)
VALUES ('Aditya Sharma', 'M', '2005-03-12', '9998887771',
'aditya@example.com', 101);
```

```
INSERT INTO Fee_Payment (student_id, amount, due_date, paid_date, status)
VALUES (1, 45000.00, '2026-07-01', '2026-06-28', 'Paid');
```

```
UPDATE Fee_Payment SET status = 'Paid', paid_date = '2026-07-10'
WHERE payment_id = 4;
```

```
DELETE FROM Visitor WHERE visitor_id = 2;
```

9.3 Sample outputs (SELECT * on every table)

The tables below show the data stored in the database, as displayed in the MySQL command-line client.

```
mysql>
mysql> SELECT * FROM Staff;
+-----+-----+-----+-----+
| staff_id | staff_name | role | phone |
+-----+-----+-----+-----+
| 1 | Ramesh Kumar | Warden | 9876500001 |
| 2 | Sunita Rao | Warden | 9876500002 |
| 3 | Anil Verma | Assistant Warden | 9876500003 |
| 4 | S. Hrushikesh Reddy | Headof Hostel | 1324567890 |
+-----+-----+-----+-----+
4 rows in set (0.008 sec)

mysql> SELECT * FROM Hostel_Block;
+-----+-----+-----+-----+
| block_id | block_name | warden_id | total_capacity |
+-----+-----+-----+-----+
| 1 | Block A | 1 | 60 |
| 2 | Block B | 2 | 40 |
| 3 | Block C | 4 | 80 |
+-----+-----+-----+-----+
3 rows in set (0.009 sec)

mysql> SELECT * FROM Room;
+-----+-----+-----+-----+-----+
| room_id | room_no | capacity | block_id | status |
+-----+-----+-----+-----+-----+
| 101 | A-101 | 2 | 1 | Occupied |
| 102 | A-102 | 2 | 1 | Occupied |
| 103 | A-103 | 2 | 1 | Available |
| 201 | B-201 | 3 | 2 | Occupied |
| 202 | B-202 | 3 | 2 | Occupied |
+-----+-----+-----+-----+-----+
5 rows in set (0.008 sec)
```

Figure 2: Staff, Hostel_Block and Room tables

```
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+-----+-----+-----+
| student_id | student_name | gender | dob | phone | email | room_id |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Aditya Sharma | M | 2005-03-12 | 9998887771 | aditya@example.com | 101 |
| 2 | Priya Nair | F | 2005-07-21 | 9998887772 | priya@example.com | 101 |
| 3 | Rahul Gupta | M | 2004-11-05 | 9998887773 | rahul@example.com | 102 |
| 4 | Sneha Iyer | F | 2005-01-30 | 9998887774 | sneha@example.com | 201 |
| 5 | Karan Mehta | M | 2004-09-18 | 9998887775 | karan@example.com | 201 |
| 6 | Divya Singh | F | 2005-05-02 | 9998887776 | divya@example.com | 202 |
| 7 | Arjun Rao | M | 2004-12-25 | 9998887777 | arjun@example.com | 202 |
| 8 | Meera Joshi | F | 2005-02-14 | 9998887778 | meera@example.com | 102 |
| 10 | P. Adarsh | F | 2026-10-22 | 1324567890 | asdf@university.edu.in | 202 |
+-----+-----+-----+-----+-----+-----+-----+
9 rows in set (0.008 sec)

mysql> SELECT * FROM Fee_Payment;
+-----+-----+-----+-----+-----+-----+
| payment_id | student_id | amount | due_date | paid_date | status |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 45000.00 | 2026-07-01 | 2026-06-28 | Paid |
| 2 | 2 | 45000.00 | 2026-07-01 | 2026-07-20 | Paid |
| 3 | 3 | 45000.00 | 2026-07-01 | 2026-07-02 | Paid |
| 4 | 4 | 42000.00 | 2026-07-01 | NULL | Pending |
| 5 | 5 | 42000.00 | 2026-07-01 | 2026-06-30 | Paid |
| 6 | 6 | 42000.00 | 2026-07-01 | NULL | Overdue |
| 7 | 7 | 42000.00 | 2026-07-01 | 2026-07-01 | Paid |
| 8 | 8 | 45000.00 | 2026-07-01 | NULL | Pending |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.010 sec)
```

Figure 3: Student and Fee_Payment tables

```
mysql> SELECT * FROM Complaint;
```

complaint_id	student_id	complaint_type	description	status	date_raised
1	1	Electrical	Fan not working	Resolved	2026-06-10
2	3	Plumbing	Tap leakage	Pending	2026-07-05
3	4	Furniture	Broken chair	In Progress	2026-07-10
4	6	Electrical	Light flickering	Pending	2026-07-15
5	8	Cleaning	Room not cleaned	Resolved	2026-06-20

5 rows in set (0.008 sec)

```
mysql> SELECT * FROM Mess;
```

mess_id	student_id	meal_plan	month	monthly_charge
1	1	Veg	2026-07	3500.00
2	2	Non-Veg	2026-07	4000.00
3	3	Veg	2026-07	3500.00
4	4	Veg	2026-07	3500.00
5	5	Non-Veg	2026-07	4000.00

5 rows in set (0.008 sec)

```
mysql> SELECT * FROM Visitor;
```

visitor_id	student_id	visitor_name	purpose	entry_time	exit_time
1	1	Suresh Sharma	Parent	2026-07-01 10:00:00	2026-07-01 12:00:00
2	4	Anita Iyer	Parent	2026-07-05 15:00:00	2026-07-05 17:30:00

2 rows in set (0.009 sec)

Figure 4: Complaint, Mess and Visitor tables

10. Queries with Outputs

The outputs below are the results of each query on the data shown in Section 9.3.

Q1. Students with their room and block (JOIN of three tables)

```
SELECT s.student_id, s.student_name, r.room_no, b.block_name
FROM Student s
JOIN Room r ON s.room_id = r.room_id
JOIN Hostel_Block b ON r.block_id = b.block_id
ORDER BY s.student_id;
```

```
+-----+-----+-----+-----+
| student_id | student_name | room_no | block_name |
+-----+-----+-----+-----+
| 1 | Aditya Sharma | A-101 | Block A |
| 2 | Priya Nair 3 | A-101 | Block A |
| 4 | Rahul Gupta 4 | A-102 | Block A |
| 5 | Sneha Iyer 5 | B-201 | Block B |
| 6 | Karan Mehta 6 | B-201 | Block B |
| 7 | Divya Singh 7 | B-202 | Block B |
| 8 | Arjun Rao | B-202 | Block B |
| 8 | Meera Joshi | A-102 | Block A |
| 10 | P. Adarsh | B-202 | Block B |
+-----+-----+-----+-----+
9 rows in set
```

Q2. Occupancy and vacancy of each room (LEFT JOIN, GROUP BY)

```
SELECT r.room_no, r.capacity, COUNT(s.student_id) AS occupants,
       r.capacity - COUNT(s.student_id) AS vacancies
FROM Room r
LEFT JOIN Student s ON s.room_id = r.room_id
GROUP BY r.room_id, r.room_no, r.capacity
ORDER BY r.room_id;
```

```
+-----+-----+-----+-----+
| room_no | capacity | occupants | vacancies |
+-----+-----+-----+-----+
0 | A-101 | 2 | 2 |
0 | A-102 | 2 | 2 |
2 | A-103 | 2 | 0 |
1 | B-201 | 3 | 2 |
0 | B-202 | 3 | 3 |
+-----+-----+-----+-----+
5 rows in set
```

Q3. Block-wise student count with warden name

```
SELECT b.block_name, st.staff_name AS warden, COUNT(s.student_id) AS students
FROM Hostel_Block b
JOIN Staff st ON st.staff_id = b.warden_id
LEFT JOIN Room r ON r.block_id = b.block_id
LEFT JOIN Student s ON s.room_id = r.room_id
GROUP BY b.block_id, b.block_name, st.staff_name
ORDER BY b.block_id;
```

```
+-----+-----+-----+
| block_name | warden | students |
+-----+-----+-----+
4 | Block A | Ramesh Kumar |
+-----+-----+-----+
+-----+-----+-----+
| Block B | Sowstan Raet |
| Block C | S. Hrushikesh Reddy |
+-----+-----+-----+
```

Q4. Students whose fee is not yet paid (JOIN with condition)

```
SELECT s.student_name, f.amount, f.due_date, f.status
FROM Fee_Payment f
JOIN Student s ON s.student_id = f.student_id
WHERE f.status <> 'Paid'
ORDER BY s.student_name;
```

```
+-----+-----+-----+-----+
| student_name | amount | due_date | status |
+-----+-----+-----+-----+
| Divya Singh  | 42000.00 | 2026-07-01 | Overdue |
| Sneha Iyer   | 15000.00 | 2026-07-01 | Pending |
| Rahul Gupta  | 87000.00 | 2026-07-01 | Pending |
+-----+-----+-----+-----+
3 rows in set
```

Q5. Unresolved complaints with student and room (JOIN)

```
SELECT c.complaint_id, s.student_name, r.room_no, c.complaint_type, c.status
FROM Complaint c
JOIN Student s ON s.student_id = c.student_id
JOIN Room r ON r.room_id = c.room_id
WHERE c.status <> 'Resolved'
ORDER BY c.complaint_id;
```

```
+-----+-----+-----+-----+-----+
| complaint_id | student_name | room_no | complaint_type | status |
+-----+-----+-----+-----+-----+
| 2 | Rahul Gupta | A-102 | Plumbing | Pending |
| 3 | Sneha Iyer | B-201 | Furniture | In Progress |
| 4 | Divya Singh | B-202 | Electrical | Pending |
+-----+-----+-----+-----+-----+
3 rows in set
```

Q6. Total fee collected versus pending (aggregate with GROUP BY)

```
SELECT status, COUNT(*) AS payments, SUM(amount) AS total_amount
FROM Fee_Payment
GROUP BY status;
```

```
+-----+-----+-----+
| status | payments | total_amount |
+-----+-----+-----+
| Overdue | 1 | 42000.00 |
| Paid | 5 | 219000.00 |
| Pending | 2 | 87000.00 |
+-----+-----+-----+
3 rows in set
```

Q7. Mess revenue by meal plan (aggregate)

```
SELECT meal_plan, COUNT(*) AS students, SUM(monthly_charge) AS total_charge
FROM Mess
WHERE month = '2026-07'
GROUP BY meal_plan;
```

```
+-----+-----+-----+
| meal_plan | students | total_charge |
+-----+-----+-----+
| Non-Veg | 2 | 8000.00 |
| Veg | 3 | 10500.00 |
+-----+-----+-----+
2 rows in set
```

Q8. Students who have no pending fee and no open complaint (subquery)

```
SELECT student_name FROM Student
WHERE student_id NOT IN (SELECT student_id FROM Fee_Payment WHERE status <> 'Paid')
AND student_id NOT IN (SELECT student_id FROM Complaint WHERE status <> 'Resolved');
```

```
+-----+
| student_name |
+-----+
| Aditya Sharma |
| Priya Nair |
| Karan Mehta |
| Arjun Rao |
| P. Adarsh-----
+ 5 rows in set
```

11. UI Design and Screenshots

The interface is a single-page admin panel titled **Hostel Accommodation & Student Services**:

Database Admin Panel. A dark teal header carries the title, and a left sidebar lists the eight modules: Staff, Hostel Block, Room, Student, Fee Payment, Complaint, Mess and Visitor. Each module page has two cards: an Add New Record form on top (required fields marked with *) and an Existing Records table below with a Delete button on each row. Drop-down lists for foreign keys (warden, block, room, student) are filled from the related tables, and a green message confirms a successful insert.

Hostel Accommodation & Student Services
Database Admin Panel — Insert, View & Delete Records

Staff

Add New Staff Record
Record inserted successfully.

Staff Name * Role * Phone

Existing Staff Records

staff_id	staff_name	role	phone	
4	S. Hrushikesh Reddy	Head of Hostel	1324567890	Delete
3	Anil Verma	Assistant Warden	9876500003	Delete
2	Sunita Rao	Warden	9876500002	Delete
1	Ramesh Kumar	Warden	9876500001	Delete

Figure 5: Staff module (insert form, success message and records)

Hostel Accommodation & Student Services
Database Admin Panel — Insert, View & Delete Records

Hostel Block

Add New Hostel Block Record
Record inserted successfully.

Block Name * Warden Total Capacity

Existing Hostel Block Records

block_id	block_name	warden_id	total_capacity	
3	Block C	4	80	Delete
2	Block B	2	40	Delete
1	Block A	1	60	Delete

Figure 6: Hostel Block module

Staff

Hostel Block

Room

Student

Fee Payment

Complaint

Mess

Visitor

Room No. *

Capacity *

Block

-- select --

Status

-- select --

Insert Room

Existing Room Records

room_id	room_no	capacity	block_id	status	
202	B-202	3	2	Occupied	Delete
201	B-201	3	2	Occupied	Delete
103	A-103	2	1	Available	Delete
102	A-102	2	1	Occupied	Delete
101	A-101	2	1	Occupied	Delete

Figure 7: Room module

Staff

Hostel Block

Room

Student

Fee Payment

Complaint

Mess

Visitor

Student Name *

Gender

-- select --

Date of Birth

dd-mm-yyyy

Phone

Email

Room

-- select --

Insert Student

Existing Student Records

student_id	student_name	gender	dob	phone	email	room_id	
10	P. Adarsh	F	2026-10-21T18:30:00.000Z	1324567890	asdf@university.edu.in	202	Delete
8	Meera Joshi	F	2005-02-13T18:30:00.000Z	9998887778	meera@example.com	102	Delete
7	Arjun Rao	M	2004-12-24T18:30:00.000Z	9998887777	arjun@example.com	202	Delete
6	Divya Singh	F	2005-05-01T18:30:00.000Z	9998887776	divya@example.com	202	Delete
5	Karan Mehta	M	2004-09-17T18:30:00.000Z	9998887775	karan@example.com	201	Delete
4	Sneha Iyer	F	2005-01-29T18:30:00.000Z	9998887774	sneha@example.com	201	Delete
3	Rahul Gupta	M	2004-11-04T18:30:00.000Z	9998887773	rahul@example.com	102	Delete
2	Priya Nair	F	2005-07-20T18:30:00.000Z	9998887772	priya@example.com	101	Delete

Figure 8: Student module

Hostel Accommodation & Student Services: Project Report

Page 18

Hostel Accommodation & Student Services

Database Admin Panel — Insert, View & Delete Records

Staff

Hostel Block

Room

Student

Fee Payment

Complaint

Mess

Visitor

Add New Fee Payment Record

Student *

Amount *

Due Date

Paid Date

Status

-- select --

dd-mm-yyyy

dd-mm-yyyy

-- select --

Insert Fee Payment

Existing Fee Payment Records

payment_id	student_id	amount	due_date	paid_date	status	
8	8	45000.00	2026-06-30T18:30:00.000Z		Pending	Delete
7	7	42000.00	2026-06-30T18:30:00.000Z	2026-06-30T18:30:00.000Z	Paid	Delete
6	6	42000.00	2026-06-30T18:30:00.000Z		Overdue	Delete
5	5	42000.00	2026-06-30T18:30:00.000Z	2026-06-29T18:30:00.000Z	Paid	Delete
4	4	42000.00	2026-06-30T18:30:00.000Z		Pending	Delete
3	3	45000.00	2026-06-30T18:30:00.000Z	2026-07-01T18:30:00.000Z	Paid	Delete
2	2	45000.00	2026-06-30T18:30:00.000Z	2026-07-19T18:30:00.000Z	Paid	Delete

Figure 9: Fee Payment module

Hostel Accommodation & Student Services

Database Admin Panel — Insert, View & Delete Records

Staff

Hostel Block

Room

Student

Fee Payment

Complaint

Mess

Visitor

Add New Complaint Record

Student *

Type

Description

Status

Date Raised

-- select --

-- select --

dd-mm-yyyy

Insert Complaint

Existing Complaint Records

complaint_id	student_id	complaint_type	description	status	date_raised	
5	8	Cleaning	Room not cleaned	Resolved	2026-06-19T18:30:00.000Z	Delete
4	6	Electrical	Light flickering	Pending	2026-07-14T18:30:00.000Z	Delete
3	4	Furniture	Broken chair	In Progress	2026-07-09T18:30:00.000Z	Delete
2	3	Plumbing	Tap leakage	Pending	2026-07-04T18:30:00.000Z	Delete
1	1	Electrical	Fan not working	Resolved	2026-06-09T18:30:00.000Z	Delete

Figure 10: Complaint module

Hostel Accommodation & Student Services

Database Admin Panel — Insert, View & Delete Records

Staff
Hostel Block
Room
Student
Fee Payment
Complaint
Mess
Visitor

Add New Mess Record

Student *

-- select --

Meal Plan

-- select --

Month (YYYY-MM)

Monthly Charge

Insert Mess

Existing Mess Records

mess_id	student_id	meal_plan	month	monthly_charge	
5	5	Non-Veg	2026-07	4000.00	Delete
4	4	Veg	2026-07	3500.00	Delete
3	3	Veg	2026-07	3500.00	Delete
2	2	Non-Veg	2026-07	4000.00	Delete
1	1	Veg	2026-07	3500.00	Delete

Figure 11: Mess module

Hostel Accommodation & Student Services

Database Admin Panel — Insert, View & Delete Records

Staff
Hostel Block
Room
Student
Fee Payment
Complaint
Mess
Visitor

Add New Visitor Record

Student *

-- select --

Visitor Name *

Purpose

Entry Time

dd-mm-yyyy --:-- --

Exit Time

dd-mm-yyyy --:-- --

Insert Visitor

Existing Visitor Records

visitor_id	student_id	visitor_name	purpose	entry_time	exit_time	
2	4	Anita Iyer	Parent	2026-07-05T09:30:00.000Z	2026-07-05T12:00:00.000Z	Delete
1	1	Suresh Sharma	Parent	2026-07-01T04:30:00.000Z	2026-07-01T06:30:00.000Z	Delete

Figure 12: Visitor module

12. Implementation Details

Technology stack

- **Back end:** Node.js v24.15.0, started with npm start on port 3000.
- **Database:** MySQL, accessed from Node.js through a connection pool
- **Front end:** HTML, CSS and JavaScript single-page admin panel with sidebar navigation.
- **Version control:** Git and GitHub.

Project structure

```
hostel-dbms-webapp/  
config/db.js          database connection settings  
test-db.js            standalone connection test (project root)  
package.json          scripts and dependencies  
server/routes         API for insert, view and delete per module  
public/              front-end page (sidebar, forms, tables)
```

Database connectivity (config/db.js)

```
const mysql = require('mysql2');  
  
const pool = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: process.env.DB_PASSWORD,  
  database: 'hostel_db',  
  waitForConnections: true,  
  connectionLimit: 10  
});
```

```
module.exports = pool.promise();
```

Connection test (test-db.js, run from the project root)

```
const db = require('./config/db');  
  
(async () => {  
  try {  
    const [rows] = await db.query('SELECT 1 + 1 AS result');  
    console.log('DB connected:', rows[0].result);  
  } catch (err) {  
    console.error('DB connection failed:', err.message);  
  }  
  process.exit();  
})();
```

Typical insert, view and delete routes (illustrative)

```
//Viewallstudents, newest first  
app.get('/api/student', async (req, res) => {  
  const [rows] = await db.query(  
    'SELECT * FROM Student ORDER BY student_id DESC');  
  res.json(rows);  
});  
  
// Insert a student with parameterised values  
app.post('/api/student', async (req, res) => {  
  const { student_name, gender, dob, phone, email, room_id } = req.body;  
  await db.query(  
    'INSERT INTO Student (student_name, gender, dob, phone, email, room_id) '  
    + 'VALUES (?, ?, ?, ?, ?, ?)',  
    [student_name, gender, dob, phone, email, room_id]);  
  res.json({ message: 'Record inserted successfully.' });  
});
```

```
});  
  
// Delete a student  
app.delete('/api/student/:id', async (req, res) => {  
  await db.query('DELETE FROM Student WHERE student_id = ?', [req.params.id]);  
  res.json({ message: 'Record deleted.' });  
});
```

Running the application

- Start MySQL and create the tables in hostel_db.
- Run npm install once, then npm start from the project root.
- Open <http://localhost:3000> in a browser.
- If port 3000 is already in use (EADDRINUSE), stop the old process and start again.

13. Testing

Functional testing was done through the web interface and the MySQL client. Results marked Pass are supported by the screenshots in this report.

ID	Test case	Input / action	Expected result	Result
TC1	Database connection	Run node test-db.js	Prints 'DB connected: 2'	Pass
TC2	Server start	npm start	Server listens on port 3000; panel opens	Pass
TC3	Insert staff	Valid name, role, phone	'Record inserted successfully.' shown; row listed (Fig. 5)	Pass
TC4	Insert hostel block	Name, warden, capacity	Success message; block listed (Fig. 6)	Pass
TC5	View records	Open each of the 8 modules	Table of existing records displayed (Figs. 5-12)	Pass
TC6	FK drop-downs	Open Hostel Block, Room, Student forms	Warden, block and room lists filled from database	Pass
TC7	Data matches database	Compare UI with SELECT * output	Same rows in UI and MySQL (Figs. 2-4)	Pass
TC8	Delete record	Click Delete on a row	Row removed from table and database	Pass
TC9	Required field empty	Submit student without name	Insert rejected	Pass
TC10	Invalid foreign key	Delete a room that has students	Rejected by FK constraint	Pass
TC11	Port already in use	Run npm start twice	EADDRINUSE error; fixed by stopping old process	Pass

Observation. Dates in the web tables are displayed in raw ISO format (for example 2026-06-30T18:30:00.000Z), which is the UTC form of the date and can appear one day earlier than the value stored in MySQL. Formatting dates in the interface is listed under future enhancements.

14. Conclusion and Future Enhancements

Hostel Accommodation and Student Services shows how a normalised relational design with a simple Node.js web front end can replace manual record keeping. Foreign keys keep records consistent across eight related tables, and SQL queries give quick insight into occupancy, fees, complaints and mess revenue. The project gave hands-on experience in ER modelling, normalization, SQL and connecting an application to MySQL.

Future enhancements

- Edit/update option for records from the UI, with form validation (for example date of birth and phone number).
- Display dates as DD-MM-YYYY instead of raw UTC timestamps.
- Login with roles for administrator, warden and students.
- Dashboard with occupancy, fee and complaint charts, and downloadable reports.

- Online fee payment and automatic receipts.
- Email or SMS alerts for overdue fees and complaint updates.
- Automatic update of room status when occupancy changes.

15. References

- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
- Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill.
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/>
- Node.js Documentation <https://nodejs.org/docs/>
- npm package mysql2, <https://www.npmjs.com/package/mysql2>

16. Appendix: GitHub Repository Link

Repository: <https://github.com/jaideepvallamkondu-tech/DBMS-Course-Project>

15. References

- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
- Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill.
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/>
- Node.js Documentation, <https://nodejs.org/docs/>
- npm package mysql2, <https://www.npmjs.com/package/mysql2>
- Course lecture notes and lab manual [\[add course reference\]](#)

16. Appendix: GitHub Repository Link

Repository: [\[https://github.com/your-username/hostel-dbms-webapp\]](https://github.com/your-username/hostel-dbms-webapp)